Backend Engineering Assignment
Notification Service

Time Estimate: 4–6 hours

Submission Deadline: 2 days from receipt

Overview

Design and implement a **Notification Service** that can send notifications to users through multiple channels. This service will be the backbone of a platform's communication system, responsible for delivering timely and reliable notifications.

Problem Statement

Your company needs a centralized notification service that can:

- Send notifications via multiple channels (Email, SMS, Push Notification)
- Handle high throughput with reliability
- Support different message priorities
- Provide visibility into notification delivery status

You are tasked with building the backend for this service.

Functional Requirements

Core Features

Feature	Description
Multi-Channel Delivery	Support at least 3 channels: Email, SMS, and In-App/Push. Each notification request specifies the target
User Preferences	Users can opt-in/opt-out of specific channels. The service must respect these preferences.
Priority Levels	Support critical, high, normal, and low priorities. Critical/high priority messages should be processed befo
Template Support	Support message templates with variable substitution (e.g., "Hello {{name}}, your order {{order_id}} has s
Delivery Tracking	Track the status of each notification: pending, sent, delivered, failed.
Retry Mechanism	Automatically retry failed notifications with exponential backoff (max 3 retries).

API Endpoints

Design and implement the following REST API endpoints:

```
POST /notifications # Send a new notification
GET /notifications/:id # Get notification status
GET /users/:userId/notifications # Get notification history for a user
POST /users/:userId/preferences # Set user channel preferences
GET /users/:userId/preferences # Get user channel preferences
```

Non-Functional Requirements

Requirement	Expectation
Scalability	Design should support 1000+ notifications/second
Reliability	No notification should be lost; use persistent queues
Idempotency	Duplicate requests with the same idempotency key should not send duplicate notifications
Rate Limiting	Implement rate limiting per user (e.g., max 100 notifications/hour per user)
Observability	Include structured logging for debugging and monitoring

Technical Constraints

- **Language:** Python
- **Database:** Use any relational (PostgreSQL preferred) or NoSQL database
- **Queue:** Use any message queue (Redis, RabbitMQ, SQS, or an in-memory implementation for demo purposes)
- **External Services:** Mock the actual email/SMS/push providers—focus on the service design, not third-party integrations

What We're Looking For

Code Quality

- Clean, readable, and well-organized code
- Consistent naming conventions and code style
- Appropriate separation of concerns (controllers, services, repositories)

System Design

- Sensible database schema design
- Proper use of queues for async processing
- Consideration for failure scenarios and edge cases

API Design

- RESTful conventions
- Proper HTTP status codes
- Clear and consistent request/response formats
- Input validation and error handling

Testing

- Unit tests for core business logic
- Integration tests for API endpoints
- Edge case coverage

Documentation

- Clear README with setup instructions
- API documentation (OpenAPI/Swagger preferred)
- Design decisions and trade-offs explained

Submission Requirements

Please submit a **GitHub repository** containing:

1. README.md

Must include:

- Project overview
- Tech stack used with rationale
- Setup instructions (local development)
- How to run tests
- API documentation or link to it
- Any assumptions made

2. DESIGN.md

A brief document (1–2 pages) covering:

- High-level architecture diagram

- Database schema with explanations
- How you handle failures and retries
- How the system would scale
- Trade-offs you made and why

3. Docker Support (Bonus)

- Dockerfile for the application
- docker-compose.yml for running with dependencies (DB, Redis, etc.)

Evaluation Criteria

Criteria	Weight	What We Assess
Functionality	25%	Does it work? Are all requirements met?
Code Quality	25%	Readability, structure, best practices
System Design	20%	Architecture decisions, scalability considerations
API Design	15%	RESTful design, error handling, validation
Testing	10%	Coverage, test quality, edge cases
Documentation	5%	Clarity, completeness, professionalism

Bonus Points

These are optional enhancements that can set your submission apart:

- **Webhook Support:** Allow clients to register webhooks for delivery status updates
- **Batch API:** Endpoint to send notifications to multiple users at once
- **Analytics Endpoint:** Basic stats (sent/failed counts by channel, time period)
- **Circuit Breaker:** Implement circuit breaker pattern for external provider calls
- **OpenAPI Spec:** Full OpenAPI 3.0 specification with examples
- **Kubernetes Manifests:** K8s deployment configurations

Clarifications & Assumptions

You may make reasonable assumptions where requirements are ambiguous. Document all assumptions in your README.

Example assumptions you might make:

- Authentication/authorization is handled by an API gateway (you can skip auth or use a simple API key)
- User data exists in a separate service; you only need to store userId
- Template storage can be in-memory or database—your choice

Questions?

If you have any clarifying questions about the requirements, please reach out to your recruiter or hiring manager. We're happy to help clarify scope.

How to Submit

Share your GitHub repository link with the hiring team.

Good luck! We look forward to reviewing your submission.